

Assessment 3: Product demonstration and post-implementation report

Giada Arosio (A00168823), Haotian Wu (A00156322),
and Mikhail Zagoruiko (A00163187)

Design and Creative Technologies, Torrens University

PBT205: Project Based Learning Studio: Technology

Fahad Hameed

May 9, 2026

Background and objectives

The growing reliance on distributed systems has made message-oriented middleware a key part of how modern software components stay scalable and loosely coupled. Following on from the Assessment 1 brief, the project set out to explore message-passing architectures with RabbitMQ acting as the central middleware between independent services.

This post-implementation review is an honest look at how the project performed against the plan agreed at the start of the semester: what was delivered, what was cut, where we missed, and what we would do differently. Its scope covers the contact tracing prototype selected as the theme to demonstrate in Assessment 2, including the work that continued on it after the April prototype freeze, and the team's working practices.

Project Scope Outcomes

From the first assessment onward the work was aimed at the requirements of the final product. In practice this meant choosing technologies that could deliver a graphical interface rather than the terminal-based prototypes the early specification allowed. The GUI was treated as a hard requirement because it is the part of the system a user actually interacts with during the demonstration.

On the functional side, the contact tracing prototype meets the brief. A user can register and sign in through the local credential flow. Once authenticated, they are placed onto a shared grid where their position is broadcast through RabbitMQ and synchronised across all connected clients in real time. The tracker service records co-location events tied to account IDs so contact between two users in the same cell can be queried later. After the April freeze the front end was widened to adapt to smaller screens, so the prototype now runs on a phone as well as a laptop.

On the technical side, the system is split into a Rust backend, a Vite/TypeScript front end, a PostgreSQL database, RabbitMQ for inter-service messaging, and Docker Compose to wire it all together for local runs. A separate tracker service handles contact recording.

Advanced features that would push the prototype toward a production-ready system were intentionally left out for time reasons: horizontal scaling of the tracker, retention policies on co-location data, and a mobile-native client. The trade-off was made explicitly rather than glossed over.

Assumptions, Risks and Constraints

Two assumptions sat under the plan. The first was that the build environment would be Linux/WSL, which kept the toolchain simple but meant a clean Windows

machine had extra setup to do. The second was that a developer running the prototype would do so locally from the repository rather than against a hosted instance, which is why so much effort went into the build scripts and the README rather than into deployment.

The main risks tracked through the semester were time, integration drift, and single-point knowledge. Time was the realistic risk throughout: with three subjects each running in parallel, running out of weeks before running out of features was always likely, and this drove the production-grade scope cuts. Integration drift: the seams between RabbitMQ, Postgres, Docker, OAuth and the front end coming apart as more was added. It was mitigated by integrating early and keeping a working compose file in main from mid-February onward. Single-point knowledge, where one team member ends up holding all the context for a given prototype, was addressed through walkthroughs at the weekly meetings.

Constraints were the usual ones for such a project: a fixed timeline and no budget for paid services or hosting.

Schedule, Deliverables, Resources and Cost

The schedule followed the rhythm of the three assessments. February was spent setting up the repository, integrating Docker, Postgres and RabbitMQ, and getting an end-to-end chat flow working. March was spent building out the contact tracing prototype on top of that foundation: the infinite grid, the position synchronisation, the tracker service, and the architecture presentation that supported Assessment 2. The prototype was marked feature-complete at the end of March.

After Assessment 2 the work on contact tracing shifted from new features to hardening and presentation, and was carried out by a single developer. On 13 April an integration test was added so the message-passing flow could be exercised end-to-end without a manual UI walkthrough. On 20 April the Assessment 2 presentation was uploaded alongside a small round of prototype tidy-ups. On 29 April the front-end build tooling (Vite) was updated. On 6 May the front end was reworked to be adaptive, with explicit support for mobile devices, so the prototype could run on phones as well as desktops. These post-freeze items closed the gap between "works on a developer's laptop" and "ready to be shown to a user."

The main deliverables were the running prototype, the architecture presentation, the README and supporting build scripts, and this report. Resources were entirely human time plus open-source software; there was no monetary cost. Total effort is estimated at roughly 90 hours from the initial commit on 11 February through to the final mobile-adaptive change on 6 May.

Quality and Test Results

Quality was approached pragmatically. Most of the build phase validation was manual: bring the stack up with Docker Compose, open two browser windows, and confirm that messages, presence updates and contact events flowed correctly between them. This was fast enough during active development that automated coverage felt premature.

Once the feature set settled, that calculation changed. The integration test added on 13 April covers the end-to-end path that previously had to be checked by hand: it spins up the relevant services and asserts that a message published by one client is observed by the other through RabbitMQ. It is not exhaustive, but it is enough to catch the regressions worth worrying about, particularly during the dependency update and UI rework that followed. The Vite update on 29 April was verified against this test and against a manual run of the full flow before being merged.

The mobile-adaptive change on 6 May was tested on three viewports: desktop, tablet and phone, by manually walking through registration, sign-in, the grid view, and a chat exchange on each. No functional regressions were found, although a handful of small visual issues at the smallest viewport were recorded as known issues rather than fixed in this round.

Open defects are limited to those cosmetic items and the deliberate scope cuts noted earlier. There is no defect that would prevent a developer from running and demonstrating the prototype.

Group Reflection

Looking back across the semester, the parts of the project that worked well were mostly consequences of decisions made early. Committing to RabbitMQ as the spine of the system before writing any service-specific logic meant that every later feature plugged into the same shape. Getting Docker Compose and Postgres running in the same week as the first chat prototype had a similar effect: by the time authentication, persistence and a message broker were needed in the same demo, none of those pieces were new.

The parts that worked less well were mostly about distribution of work and about timing. The team met weekly and discussed directions together, but the implementation hours were unevenly distributed across the three of us, and the visible commit history reflects that. A branch-and-PR convention agreed at the start would have given each member a clearer record of their technical contribution and forced more code review along the way. The pairing done on design and debugging does not show up in the log at all.

Timing was the other recurring pressure. The pattern across all three assessments was the same: a strong first half, a slightly rushed second half, and a final week dominated by polish rather than new functionality. The April and May work

on the contact tracing prototype illustrates this directly. The integration test, the Vite update and the mobile-adaptive UI were all valuable, but each is the kind of work more comfortable to do in week six than in week twelve. If the project ran again the team would freeze the feature set a fortnight earlier and use the recovered time to write tests in parallel with features.

Communication was honest, which counts as a success even though it is hard to measure. Blockers were named at the next meeting rather than hidden, and scope cuts were discussed openly. The one running disagreement was about how much polish was enough for the demonstration.

Conclusion and Lessons Learned

The project delivered what it set out to deliver. The contact tracing prototype is a working distributed system, built on a real message broker, with a usable interface that now adapts to mobile devices, an integration test covering its core flow, and a reproducible build. The scope cuts were deliberate, documented, and consistent with a limited timeline project rather than a production launch. The lessons worth carrying forward are practical: integrate early, freeze the feature set a fortnight before the deadline, write the integration test at the same time as the feature it covers.